

VertiSafe: ESP32-Enabled Elevator Control with Sensor-Based Safety Mechanisms

Alessandra R. Abuque
FAITH Colleges

s2023100044@firstasia.edu.ph

Eurylyn B. Florido
FAITH Colleges

s2023101792@firstasia.edu.ph

John Mitchell M. Fruto
FAITH Colleges

s2023101186@firstasia.edu.ph

Francine Mae R. Hirang
FAITH Colleges

s2023101786@firstasia.edu.ph

Gabrielle B. Nunez
FAITH Colleges

s2021100118@firstasia.edu.ph

Abstract - This paper presents VertiSafe, an ESP32-based elevator control system that addresses the safety and monitoring limitations of traditional elevators. Using an ESP32 CH340 microcontroller with C++ firmware, the system integrates multiple sensors including the MPU-6050 accelerometer, ACS712 current sensor, and HC-SR04 ultrasonic sensor to enable real-time monitoring of elevator position, speed, and fault conditions. Key features include SCAN scheduling for efficient floor navigation, mechanical door locking, automatic and manual emergency activation, jerk and overcurrent detection, and a WebSocket-based GUI for live telemetry. Experimental results show a mean sensor accuracy of 81.33% with an average GUI latency of 128.06 milliseconds. VertiSafe serves as a foundational prototype for safer and smarter elevator systems through embedded IoT technology.

Index Terms — ESP32, Elevator Control System, Embedded Systems, IoT Safety, SCAN Scheduling, Real-Time Monitoring, WebSocket, Emergency Response

INTRODUCTION

Elevator systems are vital for high-rise buildings, yet traditional models only offer basic call functions and limited safety indicators. Breakdowns often trap passengers due to poor tracking and monitoring. Modern elevators require precise performance, user movement verification, and continuous monitoring, but current systems lack sensor-based intelligence, increasing risks [1,2]. Integrating embedded systems and IoT enables real-time feedback, sensor control, and safer, more efficient operations through microcontroller-based technology [3,4,5,6].

The project develops VertiSafe, an elevator control system based on ESP32 technology which improves safety, system reliability, and emergency response capabilities. The system uses a mechanical door locking system which secures doors during both regular operations and emergency situations, while a winch-driven drum system enables vertical transportation. Emergency response occurs through two methods which include manual activation by passengers through a dedicated button, and automatic system detection of sudden jerks of MP-6050 and overcurrent spikes of ACS712. The system tracks motion through three different sensors which include an ultrasonic sensor, a tachometer, and an accelerometer while its requests operate under SCAN scheduling which rules together with direction-priority queuing systems. The system provides three additional features which include a passive buzzer for alerts, energy consumption tracking, and automatic queue reordering all of which users can monitor through a graphical user interface in real time.

The study is limited to a laboratory prototype that does not extend to commercial elevator operations or meet international safety standards. The system lacks vibration sensors, preventing detection of misalignment and excessive vibration. Emergency systems require manual activation, except for automatic jerk and overload detection with nearest floor routing. Several features remain unimplemented, including PWM-based speed control, weight-based overload protection, and elimination of steady-state error for precise floor alignment. Performance is restricted compared to commercial systems, though future work includes vibration-based fault detection, full emergency automation, and scalability testing. Additionally, WiFi credentials are hardcoded in plaintext, which poses a security vulnerability.

The research shows that real-time monitoring combined with embedded control systems creates better results for elevator safety. The study by Ozdemir et al. [1] established that anomaly detection helps track traction faults, while Sanfilippo et al. [7] proved that state-machine logic successfully controls elevator movement. An et al. [2] developed an IoT multi-sensor system that detects hazardous conditions, and Rubies et al. [3] together with Tan [5] investigated ESP32 and Arduino systems for contactless safety-based elevator operation. The research by Khanna et al. [8] and Wang [6] showed that organizations should move from using old relay systems toward adopting programmable controllers and AI-based monitoring. Lai et al. [9] confirmed that sensor-based remote-control technology enables faster emergency responses, and Sarayodhin et al. [10] showed how advanced maintenance strategies with embedded safety systems create a dependable operational system.

The significance of the study shows how VertiSafe safely protects passengers during emergency situations which occur in traditional elevator systems. The system uses its mechanical locking mechanism together with sensor intelligence and scheduling optimization to deliver precise real-time information about its current movement and operational state. The graphical user interface enables users to see operations through visual monitoring while the system protects passengers through both manual and automatic emergency activation methods. VertiSafe achieves operational safety through its embedded systems combined with feedback control and sensor technology and mechanical design elements.

METHODOLOGY

VertiSafe is designed with a sensor-driven and safety-focused approach using ESP32 technology. Ultrasonic and motion sensors provide real-time monitoring of elevator movement and floor position, while a mechanical lock and emergency algorithm ensure passenger safety during faults or emergencies. The SCAN scheduling method further improves efficiency by reducing unnecessary reversals. Overall, the system combines sensors, mechanical safety, and optimized control to create a reliable and secure elevator prototype.

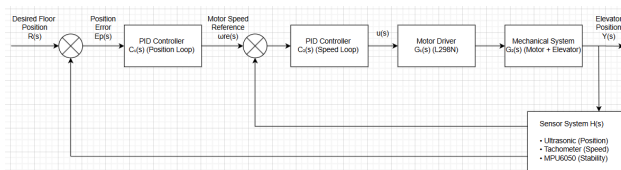


FIGURE 1.1. BLOCK DIAGRAM OF VERTISAFE.

Figure 1.1 shows a dual-loop feedback system with a position controller $C1(s)$ that converts floor-level error

into speed reference values while speed controller $C2(s)$ controls motor actuation. The motor driver $G1(s)$ provides gain with electrical delay which connects to the mechanical system $G2(s)$. The mechanical system $G2(s)$ calculates torque, inertia, and damping. Sensor feedback $H(s)$ ensures the system monitors position, speed, and stability.

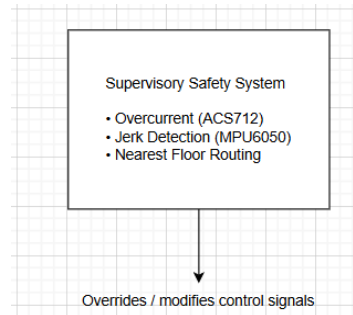


FIGURE 1.2. ADDITIONAL SUPERVISORY SAFETY SYSTEM.

In figure 1.2, it shows that the system activates its independent supervisory safety system during faults which include overcurrent and excessive jerk to direct the cabin to the nearest floor. The current ESP32 prototype operates with open-loop motor control and sensor monitoring rather than the intended dual-loop PID design. The system will implement PID feedback through future upgrades. The system uses its two degrees of freedom to achieve precise control while the dual-loop structure and supervisory mechanism provide additional safety measures.

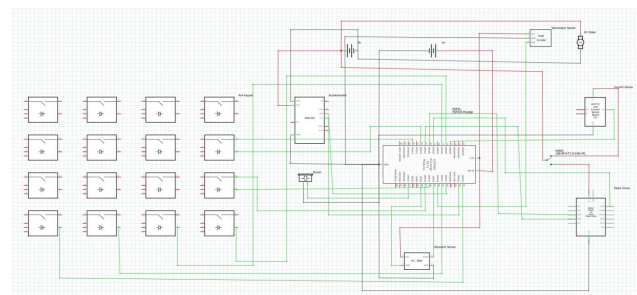


FIGURE 2. SCHEMATIC DESIGN OF VERTISAFE.

Figure 2 illustrates how VertiSafe connects its hardware components into a functional elevator control system. At the core, the ESP32 CH340 microcontroller manages inputs from the 4×4 keypad, ultrasonic sensor for floor detection, tachometer for motor RPM, MPU-6050 accelerometer for jerk monitoring, and ACS712 current sensor for overload detection. It controls outputs to the L298N motor driver, which enables bidirectional motor movement while protecting GPIO pins, as well as the DC motor, door mechanism, and buzzer for alerts. The prototype structure uses a winch

mechanism with a metal axle, bearing, and spacers to ensure stable vertical motion. Overall, the schematic integrates sensing, actuation, and mechanical safety into a unified embedded system capable of monitoring operations, detecting faults, and executing emergency responses.

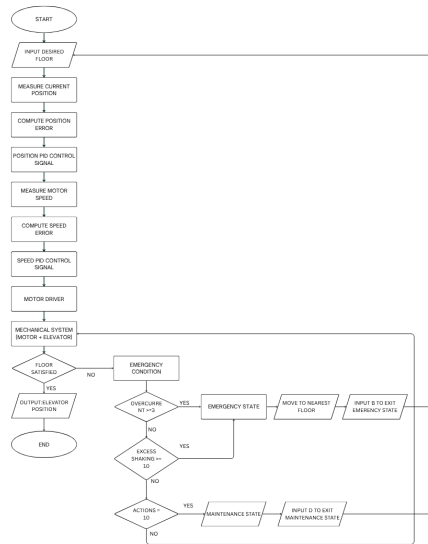


FIGURE 3. FLOWCHART.

Figure 3 shows the flowchart of the dual-loop elevator control system, where the input $R(s)$ is processed through position and speed controllers to generate the control signal $u(s)$, producing the output elevator position $Y(s)$ with continuous sensor feedback. The system monitors for emergency conditions such as excessive shaking or current overloading occurring three times, which triggers an override that moves the elevator to the nearest floor and enters a safe state, then, this emergency state can be manually exited using Button B. Additionally, a maintenance mode is activated after ten operation cycles and can be exited using Button D, while normal operation continues if no fault or maintenance condition is detected.

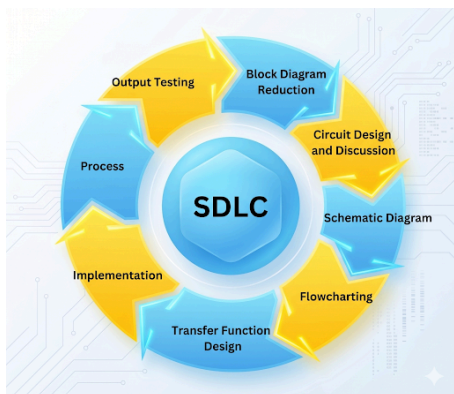


FIGURE 4. SOFTWARE DEVELOPMENT LIFE CYCLE.

The Software Development Life Cycle (SDLC) for VertiSafe uses an organized process which repeats its steps to achieve precise results through trustworthy operations. The project starts with block diagram reduction work which enables engineers to create hardware systems through circuit design and schematic development. The system implements operational logic through flowcharting and transfer function modeling which leads to the development of an ESP32-based prototype with built-in sensors. The process ends with testing and output evaluation which check three factors: system accuracy, response time, and safety override functionality.

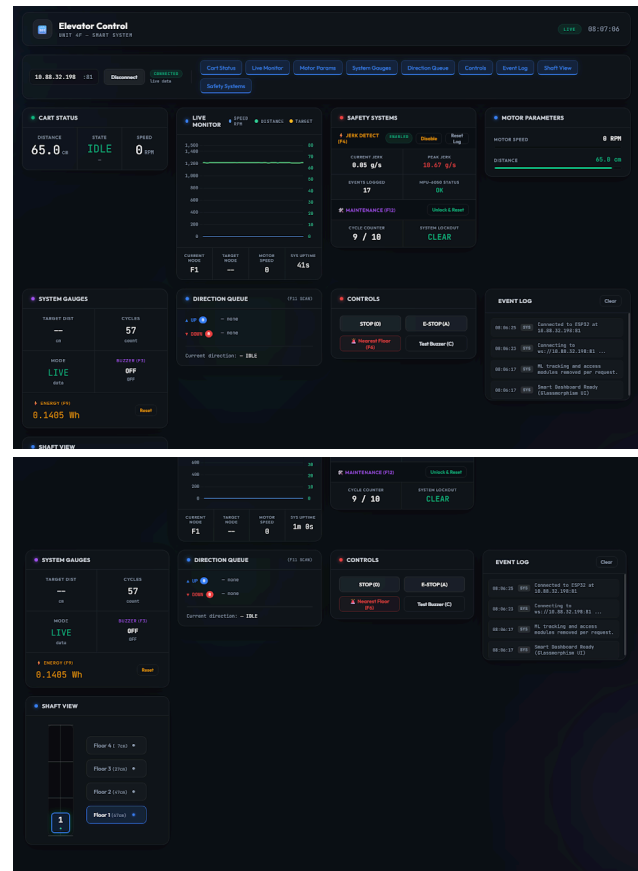


FIGURE 5. REAL-TIME DATA EXCHANGE BETWEEN ESP32-CH340 AND GRAPHICAL USER INTERFACE.

The dashboard establishes a connection to the ESP32 through WebSocket which operates on port 81 and uses the browser API to create a permanent bi-directional connection. The ESP32 establishes a connection which allows it to transmit JSON packets that contain real-time elevator information about distance, speed, motor state, and safety alerts to the dashboard which transforms the data into display interfaces and graphical elements. User actions, such as floor selection, generate JSON commands, such as `{command:'go_floor', floor:2,`

distance:47}, that are sent back through the same connection for execution. The dashboard enters simulation mode when no WebSocket connection exists which simulates elevator movement through the browser by using scheduled updates.

RESULTS AND DISCUSSION

I. Program Implementation

The VertiSafe system runs on an ESP32 microcontroller with a modular C++ framework under an SDLC process. Floor navigation uses a SCAN scheduling algorithm with direction-priority queuing and dynamic reordering. Motor control applies direct high-state digital logic for fast response. Safety routines include jerk detection via MPU-6050 accelerometer and electrical anomaly detection through ACS712 current sensor with spike filtering, triggering emergency overrides to the nearest floor. A web-based GUI manages telemetry and user commands using WebSocket for real-time JSON exchange.

```
if (travelDirection == 1 && dist < distance && dist >
targetDistance) {
    upQueue[upCount++] = {targetFloor,
targetDistance};
    targetFloor = floor;
    targetDistance = dist;
}
```

This logic implements the SCAN algorithm by intercepting new floor requests that fall within the current travel path, optimizing system efficiency.

```
digitalWrite(IN1, HIGH);
digitalWrite(IN2, LOW);
digitalWrite(ENA, HIGH); // ENA hardwired HIGH
for immediate response
```

The motor is driven via direct digital signals to the L298N module, maintaining the Enable A pin at a constant high state for maximum torque and simplified logic.

```
for (int i = 0; i < NUM_SAFE_FLOORS; i++) {
    float diff = abs(SAFE_FLOORS[i].dist - distance);
    if (diff < bestDiff) { bestDiff = diff; bestIdx = i; }
}
```

This routine calculates the absolute distance to all predefined safe floor coordinates to identify the nearest exit point during a system fault.

```
float jz = (accelZ - prevAZ) / dt;
jerkMagnitude = sqrt(jx * jx + jy * jy + jz * jz);
if (jerkMagnitude > JERK_THRESHOLD)
    triggerEmergency("JERK");
```

The system computes the derivative of acceleration (jerk) across three axes to detect mechanical instability and trigger an automated safety shutdown.

```
float amps = abs(voltage - ACS712_ZERO_V) /
ACS712_SENS;
if (amps > OC_THRESHOLD_A) { ocSpikeCount++;
}
```

A current-sensing routine monitors electrical load and utilizes a spike-filtering counter to distinguish between transient noise and genuine motor stalls.

```
StaticJsonDocument<512> doc;
doc["pos"] = distance;
doc["motor"] = motorState;
String output; serializeJson(doc, output);
websocket.broadcastTXT(output);
```

Real-time system parameters are serialized into JSON format and broadcasted via WebSockets to provide low-latency telemetry for the remote dashboard.

II. Program Output

Symbol	Description	Unit
X_{exp}	Expected (theoretical) value	varies
X_{act}	Measured sensor value	varies
X_{gui}	GUI-displayed value	varies
E_{sensor}	Sensor error	same as measured
E_{gui}	GUI error	same as measured
E_{total}	Total system error	same as measured

E_{wh}	Energy Consumption	varies
T_{sensor}	Sensor response time	s
T_{gui}	GUI update time	s
ΔT	GUI delay	s

TABLE 2. DEFINED VARIABLES AND NOTATION.

Defines the variables and formulas used to evaluate system performance, including sensor error, GUI error, total error, percentage error, accuracy, energy consumption, and GUI delay. These standardized metrics provide a consistent framework for measuring and comparing results across all trials.

$E_{sensor} = X_{act} - X_{exp} $
$E_{gui} = X_{gui} - X_{act} $
$E_{total} = X_{gui} - X_{exp} $
$\%Error = \frac{ X_{act} - X_{exp} }{X_{exp}} \times 100$
$Accuracy(\%) = 100 - \%Error$
$E_{wh} = I \times V \times t$
$\Delta T = T_{gui} - T_{sensor}$

TABLE 3. PERFORMANCE METRICS.

Presents the mathematical formulas used to calculate each performance metric defined in Table 2. These equations quantify the difference between expected, actual, and GUI-displayed values to objectively assess system accuracy and reliability.

Trial	X_{exp}	X_{act}	X_{gui}	E_{sensor}	E_{gui}	E_{total}	%E	%
1.1	65	65.8	65.8	0.8	0	0.8	1.23	98.77
1.2	65	65.6	65.6	0.6	0	0.6	0.92	99.08
2.1	45	46.4	46.4	1.4	0	1.4	3.11	96.89
2.2	45	44.	44.	0.8	0	0.8	1.78	98.22

		2	2					
3.1	27	26.6	26.6	0.4	0	0.4	1.48	98.52
3.2	27	26	26	1.0	0	1.0	3.70	96.30
4.1	5	7.1	7.1	2.1	0	2.1	42	58
4.2	5	7.7	7.7	2.7	0	2.7	54	46

TABLE 4.1. ULTRASONIC SENSOR ACCURACY IN CENTIMETERS AND GUI CONSISTENCY.

The results show that the GUI values exactly match the actual values, which is why the GUI error is zero for all trials. However, accuracy decreases significantly when the difference between the expected and actual values increases, especially in trials 4.1 and 4.2 where the percent error is very high.

Trial	X_{exp}	X_{act}	E_{sensor}	%E	%
1.1	12	12.96	0.96	8	92
1.2	12	13.51	1.51	12.58	87.42
2.1	12	13.51	1.51	12.58	87.42
2.2	12	13.51	1.51	12.58	87.42
3.1	12	13.51	1.51	12.58	87.42
3.2	12	10.17	1.83	15.25	84.75
4.1	12	12.53	0.53	4.42	95.58
4.2	12	13.51	1.51	12.58	87.42

TABLE 4.2. CURRENT SENSOR ACCURACY IN AMPERES AND GUI CONSISTENCY.

The results show that most trials have moderate errors, with Trial 4.1 having the highest accuracy due to the smallest deviation from the expected value. Meanwhile, Trial 3.2 has the lowest accuracy because it has the largest difference between the expected and actual current readings.

Trial	Voltage (V)	Current (A)	Time (s)	Energy (wh)
1.1	0	13.51	Undefined	0.0235
1.2	0.41	11.31	94.74	0.1218
2.1	0.26	12.10	82.91	0.0725

2.2	0	13.51	Undefined	0.1338
3.1	0.40	10.73	71.87	0.0857
3.2	0.19	12.49	223.13	0.1472
4.1	0.17	12.61	167.67	0.0992
4.2	0	13.51	Undefined	0.1609

TABLE 4.4. ENERGY CONSUMPTION MEASUREMENT.

The time was computed using the formula in Table 3, which relates energy, voltage, and current. Trials with zero voltage resulted in undefined time values because no electrical power is present, making the calculation impossible.

Metric	Value
Mean Accuracy	81.33%
Mean Sensor Error	1.33 UNITS
Peak Error	2.7CM
Standard Deviation	15.76%
Average GUI Delay	128.06s

TABLE 4.5. SUMMARY METRICS.

Summarizes the overall system performance, showing a mean accuracy of 81.33%, a peak error of 2.7 cm, and an average GUI delay of 128.06 seconds. These figures highlight the system's general reliability while also revealing areas that require improvement, particularly in sensor precision and response latency.

CONCLUSION

VertiSafe successfully demonstrates the feasibility of an ESP32-based elevator control system that integrates sensor-driven safety mechanisms, real-time monitoring, and scheduling optimization into a functional laboratory prototype. The system achieves its core objectives of detecting emergency conditions (jerk and overcurrent), routing the elevator to the nearest safe floor during faults, and providing real-time telemetry through a WebSocket-connected GUI. Ultrasonic sensor accuracy averaged above 96% for mid-to-upper floors, though significant error emerged at close range. Current sensing maintained a mean accuracy of 81.33% with a standard deviation of 15.76%, indicating room for improvement. Despite operating under open-loop motor control rather than the intended PID design, the system fulfilled its primary safety and monitoring goals. Future work should

prioritize closed-loop PID implementation, physical limit switches, load-cell-based overload detection, WiFi credential security, and compliance testing against established elevator safety standards before any real-world deployment is considered.

RECOMMENDATIONS

The Smart Elevator Control System requires targeted improvements to enhance operational effectiveness, safety, and reliability. On the hardware side, the absence of physical limit switches at the shaft boundaries must be addressed by integrating top and bottom limit switches connected to interrupt pins, along with independent braking mechanisms to prevent overtravel in case of sensor failure. The current-based overload detection should be replaced with a load cell-based cable tension sensor for more accurate load measurement, while the addition of a counterweight system with separate drum operation would improve load balancing, reduce motor stress, and increase energy efficiency. Installing vibration sensors would also enable detection of shaft misalignment and excessive mechanical oscillations. On the control side, proper PWM motor speed control through ENA pin activation and the implementation of closed-loop PID control using ultrasonic and tachometer feedback are necessary to achieve smooth acceleration and precise floor positioning. The current use of hardcoded floor distances (67, 47, 27, and 7 cm) should be replaced with a keypad-triggered calibration mode that stores values in ESP32 non-volatile storage, allowing reconfiguration without firmware modification. Energy estimation should be improved by computing real-time power consumption using ACS712 current readings, motor voltage, and elapsed time instead of relying on a fixed 12.0 W assumption. In terms of safety and reliability, the SCAN queue logic should incorporate a freshness check (e.g., within 100 ms) to avoid decisions based on stale distance data, while sensor fusion combining current, jerk, and position inputs would enhance fault detection accuracy. WiFi credentials, currently hardcoded in plaintext, should be secured using ESP32 provisioning or protected storage. Finally, the web-based GUI requires optimization to reduce latency under high WebSocket traffic, and the system must undergo scalability testing across varying shaft configurations and loads, as well as compliance evaluation with established elevator safety standards prior to real-world deployment.

REFERENCES

- [1] S. Ozdemir, O. N. Neamah, and R. Bayir, "Enhanced safety and efficiency in traction elevators: A real-time monitoring system with anomaly detection," *Artificial Intelligence and Autonomous Systems*, vol. 2, no. 1, Article 0001, 2025.

[2] Z. An, Y. Zhang, C. Liu, X. Li, and Y. Wang, "Intelligent monitoring system of elevator Internet of Things," *J. Phys.: Conf. Ser.*, vol. 1883, no. 1, p. 012089, 2021.

[3] E. Rubies, R. Bitriá, E. Clotet, and J. Palacín, "Non-contact and non-intrusive add-on IoT device for wireless remote elevator control," *Sensors*, vol. 23, no. 3, p. 1469, 2023.

[4] A. N. Z. Rashed, M. Yarrarapu, R. T. Prabu, et al., "Connected smart elevator systems for smart power and time saving," *Scientific Reports*, vol. 14, Article 19330, 2024.

[5] D. Tan, "Development of smart elevator using embedded system," *Int. J. Eng. Adv. Technol.*, vol. 11, no. 3, pp. 45–50, 2022.

[6] Y. Wang, "AI in functional safety systems – smart elevator monitoring," M.S. thesis, Hong Kong Polytechnic Univ., 2024.

[7] F. Sanfilippo, M. Økter, T. Eie, and M. Ottestad, "Teaching motion control in mechatronics education using an open framework based on the elevator model," in *Proc. IEEE Global Engineering Education Conf. (EDUCON)*, 2022, pp. 976–6562.

[8] H. Khanna, J. Kaur, K. Pilania, and S. Jamwal, "Revolutionizing vertical transport: A comprehensive study on the evolution and optimization of elevators," *Int. J. Eng. Res. & Technol.*, vol. 13, no. 5, pp. 210–216, 2024.

[9] S.-C. Lai, Y.-H. Chen, Y.-H. Lin, and C.-H. Huang, "Remote-control system for elevator with sensor technology," *Sensors and Materials*, vol. 34, no. 9, pp. 3251–3262, 2022.

[10] J. Sarayodhin, W. Kongsong, C. Pooworakulchai, and K. Kitiyanun, "Trends in elevator maintenance strategies: An in-depth investigation," *Int. J. Adv. Sci., Eng. Inf. Technol.*, vol. 15, no. 4, pp. 1187–1194, 2025, doi: 10.18517/ijaseit.15.4.

ABOUT THE AUTHORS



Alessandra R. Abuque is a third-year Bachelor of Science in Computer Engineering student at FAITH Colleges. She has a strong interest in joining academic and technical competitions that challenge her knowledge and creativity. She is also passionate about public speaking and continuously strives to enhance her communication and leadership skills.



Eurylyn B. Florido is a third-year Bachelor of Science in Computer Engineering student at FAITH Colleges. She is eager to learn new concepts and continuously gain experiences, believing that lifelong learning is the most effective way to improve herself both personally and professionally. She has a strong interest in mind games such as chess, which enhance her strategic thinking and problem-solving skills.



John Mitchell M. Fruto is a third-year Bachelor of Science in Computer Engineering student at the First Asia Institute of Technology and Humanities. He has a strong interest in developing his technical skills in software programming through seminars and competitive events such as hackathons. In addition to his academic pursuits, he is passionate about community involvement and previously served as the 2022 City Youth 11th Councilor of Lipa City, Batangas.



Francine Mae R. Hirang is currently taking Bachelor of Science in Computer Engineering and is in her third year at FAITH Colleges. She chose this program due to her strong interest in learning and exploring both software and hardware technologies. During her second year at FAITH Colleges, she received the Top 1 Award based on the overall evaluation of the workshop speakers for the Second-Year Workshop on C++ Programming and Database Integration.



Gabrielle B. Nunez is a fourth-year Bachelor of Science in Computer Engineering student at FAITH Colleges. He chose the Computer Engineering program for its strong integration of hardware and software, aligning with his interests in embedded systems, microcontroller-based design, and computer hardware architecture. Outside of his academic work, he has a strong personal interest in personal computers, particularly in building, and experimenting with computer hardware.